Compiladores e Intérpretes

Tema 4.6 – Analizador Semántico – Chequeo de Sentencias

Universidad Nacional del Sur

Departamento de Ciencias e Ingeniería de la Computación

2026

Chequeo de Sentencias

- El **Chequeo de Sentencias** trabaja sobre el cuerpo de las las unidades (métodos/constructores)
  - Chequeando que no tenga **errores** semánticos y
  - Recopilando información de que **entidades utiliza** y como lo hace.

- Está centrado en el **chequeo de tipos** y la **resolución de nombres** en sentencias y expresiones

Resolución de Nombres

Resolución de Nombres

- La **resolución de nombres** se refiere a que cuando se utiliza un **Identificador** en una expresión o sentencia, se controle que sea el nombre de la entidad adecuada.

Resolución de Nombres

- La clave del proceso de **resolución de nombres** es como **buscar** los **nombres** en la **tabla de símbolos**
- Para eso la **tabla de símbolos** debe proveer las herramientas de búsqueda adecuadas
- Por ejemplo, en las expresiones:

4+x

- Busco a **x** en la tablas de variables locales y parámetros del método actual.
- Si no esta en esas tablas, busco **x** en la tabla de atributos de la clase actual.
- Sino ERROR!

Resolución de Nombres

- La clave del proceso de **resolución de nombres** es como **buscar** los **nombres** en la **tabla de símbolos**
- Para eso la **tabla de símbolos** debe proveer las herramientas de búsqueda adecuadas
- Por ejemplo, en las expresiones:

  4+x

  10*m1(7)

  - Busco a **m1** en la tabla de métodos de la clase actual.
  - Sino esta ERROR!

Resolución de Nombres

- Entonces el **objetivo** de la **resolución de nombres** es controlar que un **nombre** en cierto **contexto** refiere a la **entidad adecuada**
  - Que sea la **entidad adecuada** depende de "**donde**" busquemos en la **tabla de símbolos**
  - Si **no esta en la tabla** que corresponde se produce un **error semántico**
- El **otro objetivo** de la **resolución de nombres** es obtener la **entrada de la entidad** para poder efectuar:
  - Chequeo de Tipos
  - Traducción

Chequeo de Tipos

Chequeo de Tipos

- El **chequeo de tipos** es el encargado de hacer cumplir las **restricciones impuestas** por las **reglas del sistema de tipos** del lenguaje
  - Estas **reglas** en parte son las que **indican** como se pueden **utilizar las entidades tipadas** dentro de las sentencias del lenguaje
- Es decir, controlar que cuando un **elemento tipado** (variable, método, expresión) es **utilizado** en un contexto su **tipo es el adecuado**
  - Si un elemento tipado **no tiene el tipo esperado** en el contexto utilizado se produce un **error de tipos**

Chequeo de Tipos

- En el **chequeo de tipos** es donde controlamos si los **operadores** utilizados en una operación **son de tipos validos**
- Por lo tanto, donde aplicaremos/consideraremos las **reglas de tipos** del lenguaje vinculadas a:
  - Sobrecarga
  - Coerción
  - Equivalencia/Compatibilidad
  - Polimorfismo

Chequeo de Tipos

- Como vieron en lenguajes el **chequeo de tipos** puede realizarse de **dos maneras** (no necesariamente disjuntas):
  - **Chequeos Estáticos**: controles que se realizan en **tiempo de compilación**, durante el análisis semántico
  - **Chequeos Dinámicos**: controles que se **realizan en ejecución**, donde el compilador genera código para realizar los cheques
- En esta **materia nos centraremos** en los **chequeos estáticos**.

Chequeo de Tipos

- Para poder realizar sus **tareas** de manera **estática** el chequeo de tipos **requiere** que los **tipos de los elementos**:
  - Estén **declarados**, o
  - Sean **inferibles** estáticamente
- Por ejemplo, en **MiniJava**:
  - Los tipos estáticos de **métodos, atributos y parámetros** se **declaran**
  - Los tipos estáticos de las **expresiones** y **variables locales** se **infieren** estaticamente

Chequeo de Tipos

- En general el **chequeo de tipos**, implica chequear que:
- Los tipos de la **expresión** usada en una **sentencia** sean los requeridos

Por ejemplo, en MiniJava:

- El tipo de la expresión de la condición un **if** sea booleano
- El tipo la expresión del lado derecho de una **asignación** sea igual o un subtipo del tipo de la variable correspondiente con el identificador al final de la cadena del lado izquierdo

Chequeo de Tipos

- En general el **chequeo de tipos**, implica chequear que:
- Los tipos de las **sub-expresiones** en una **expresión** sean los adecuados.
- Por ejemplo, en MiniJava:
  - El tipo de las sub-expresiones de una expresión binaria con operador ‘+’ sea **int**
  - El tipo de la sub-expresión de una expresión unaria con operador ‘!’ sea **boolean**

Chequeo de Tipos y Resolución de Nombres

- En las **expresiones** y **sentencias** donde debemos realizar **chequeos de tipos** pueden aparecer **nombres**
- Al realizar el **chequeo de tipos** debemos resolver a que entidades hacen referencia eso **nombres** para **obtener su tipo** y **controlar** que se esta **usando de forma adecuada**
- Por lo tanto, la **resolución de nombres** se suele **realizar en conjunto con** (o como parte de) el **chequeo de tipos**

Chequeo de Sentencias sobre ASTs

Chequeo Sentencias – Dos Pasadas – ASTs

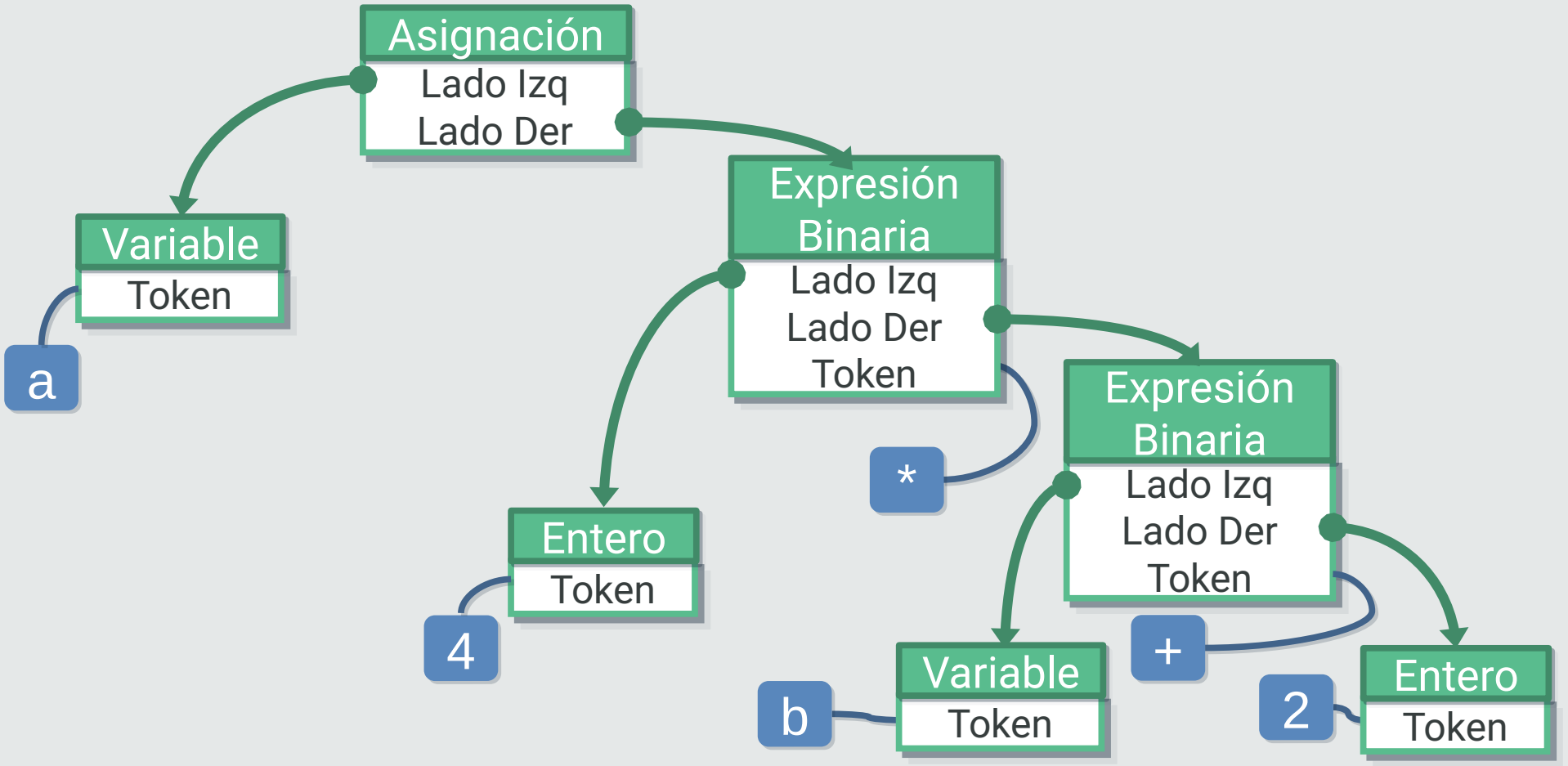
- Al **diseñar** los **nodos del AST** seguimos la **intuición**:

  "Cada **tipo de nodo** se corresponde con un **constructor sintáctico** que tiene **significado**"

- Por lo tanto, los **controles semánticos** van a estar **asociados** a cada **tipo de nodo**
- Intuitivamente entonces el **chequeo de sentencias** se realizará al **visitar** adecuadamente los **nodos del AST**

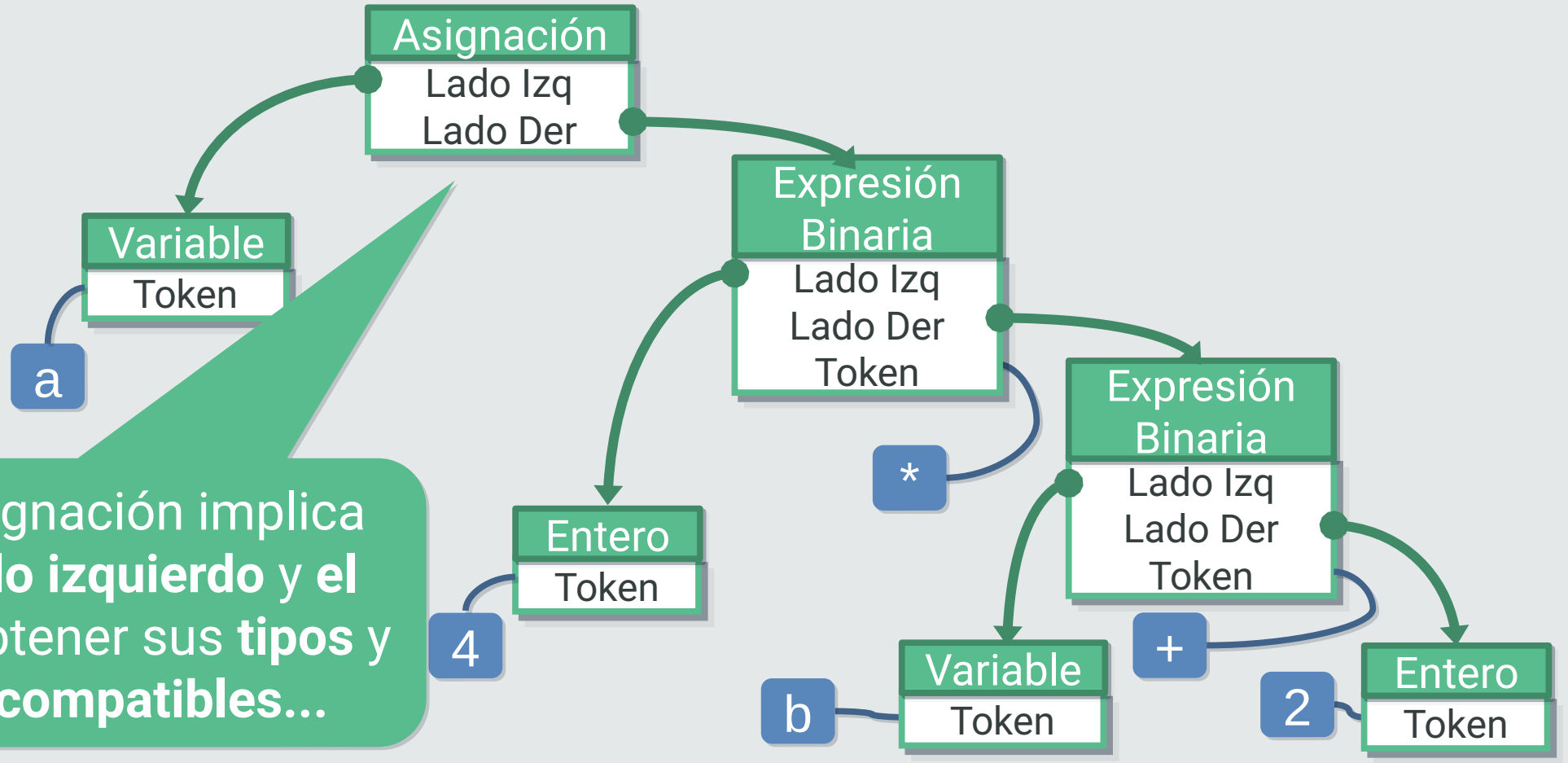
Chequeo Sentencias – ASTs – Ej. Controles

```
…
a = 4*(b+2);
…
```



Chequeo Sentencias – ASTs – Ej. Controles

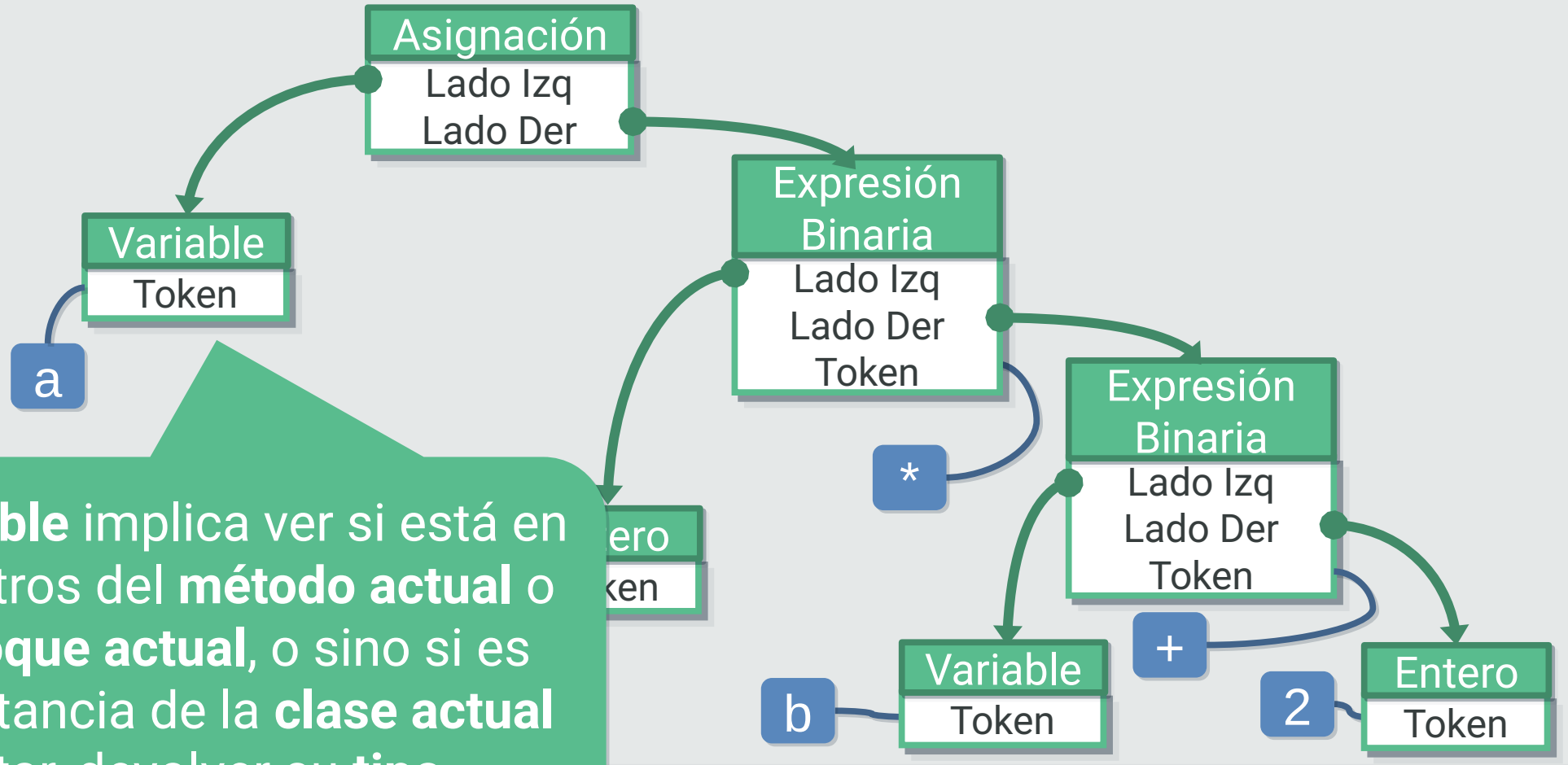
```
…
a = 4*(b+2);
…
```



Chequear la asignación implica *chequear* el **lado izquierdo** y **el lado derecho**, obtener sus **tipos** y ver que sean **compatibles...**

Chequeo Sentencias – ASTs – Ej. Controles

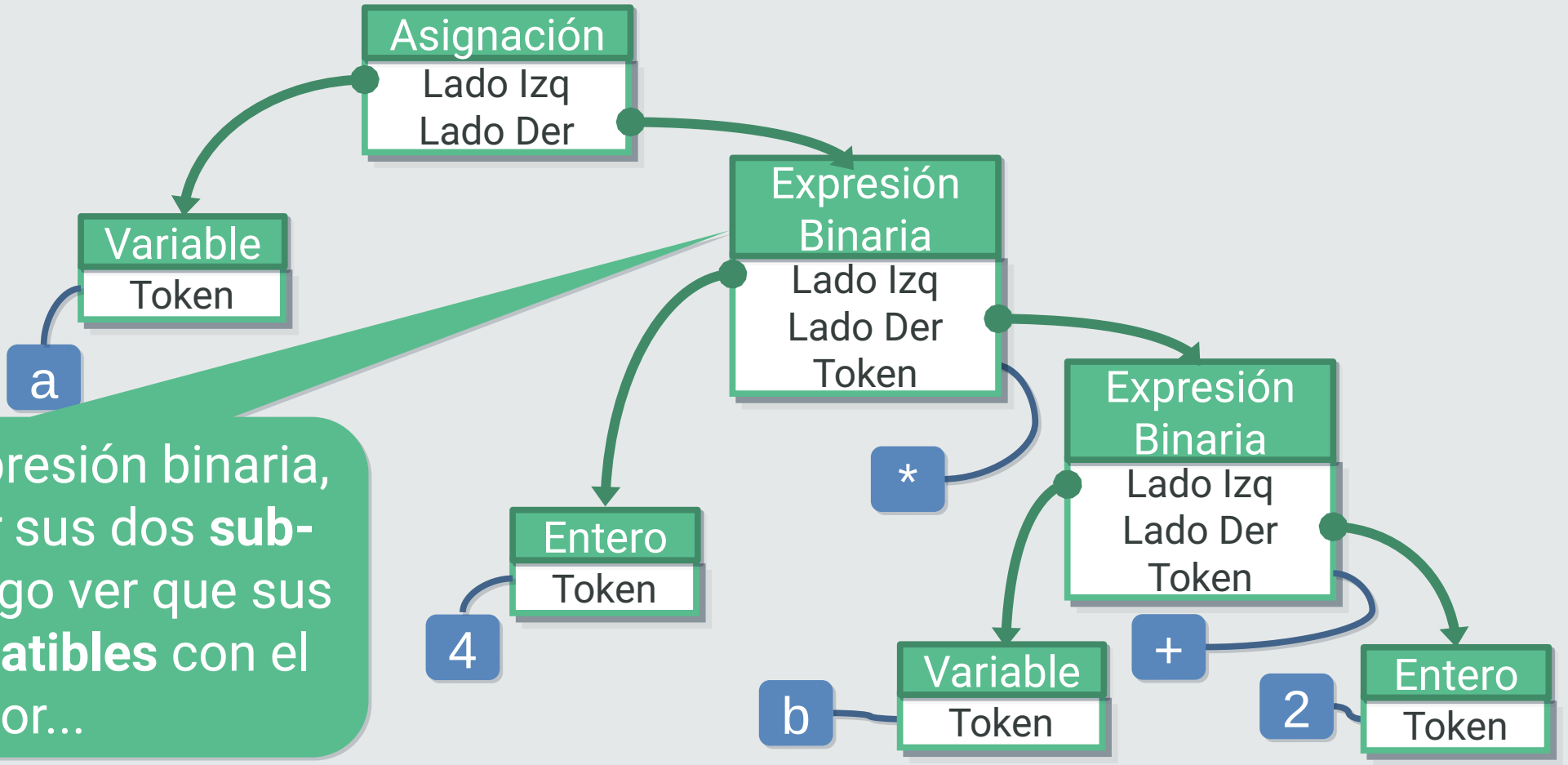
```
…
a = 4*(b+2);
…
```



Chequear una **Variable** implica ver si está en la **tabla** de parámetros del **método actual** o var locales del **bloque actual**, o sino si es una variable de instancia de la **clase actual** y, en caso de estar, devolver su **tipo**...

Chequeo Sentencias – ASTs – Ej. Controles

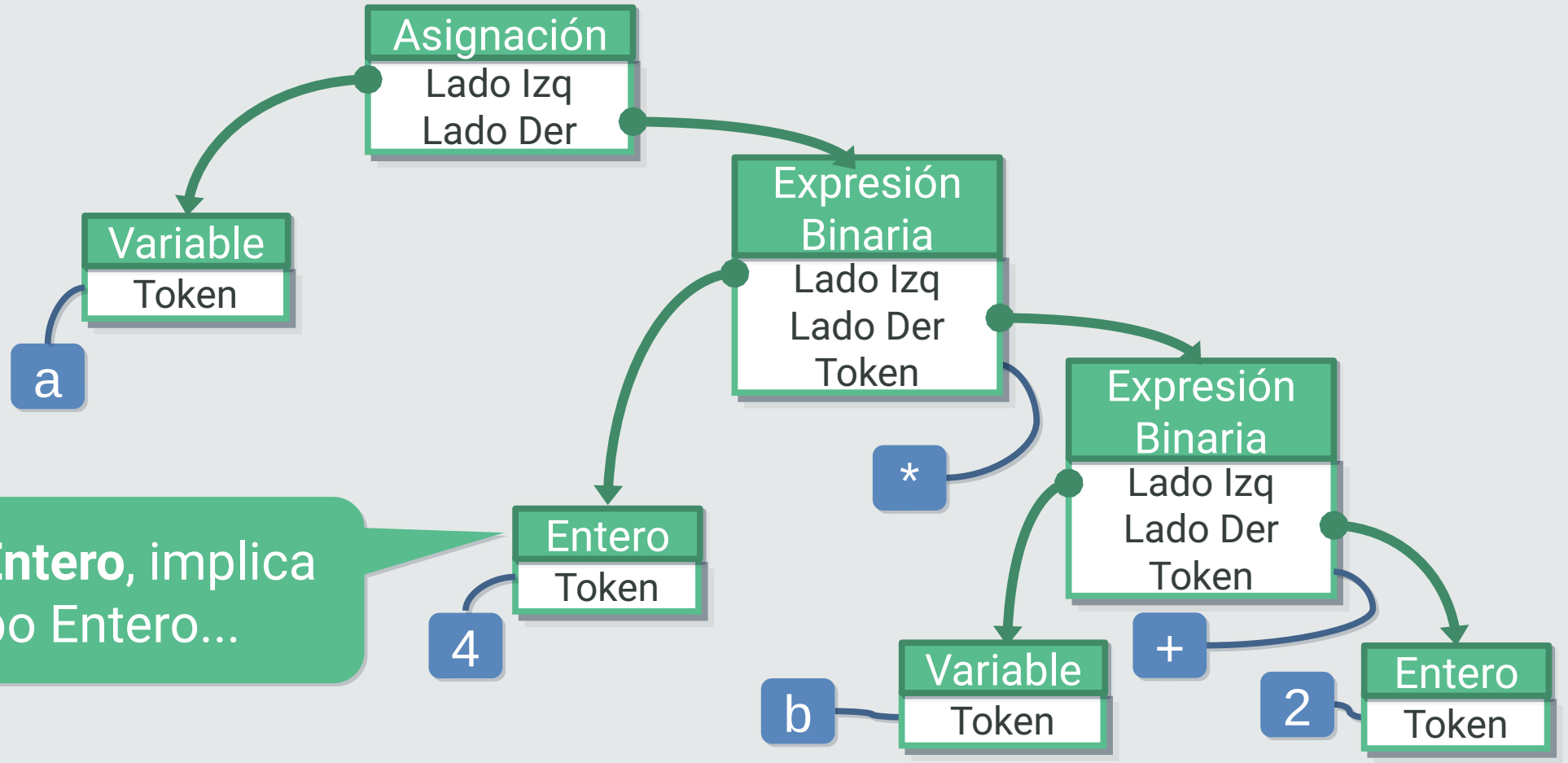
```
…
a = 4*(b+2);
…
```



Chequear una expresión binaria, implica *chequear* sus dos **sub-expresiones** y luego ver que sus **tipos** sean **compatibles** con el operador...

Chequeo Sentencias – ASTs – Ej. Controles

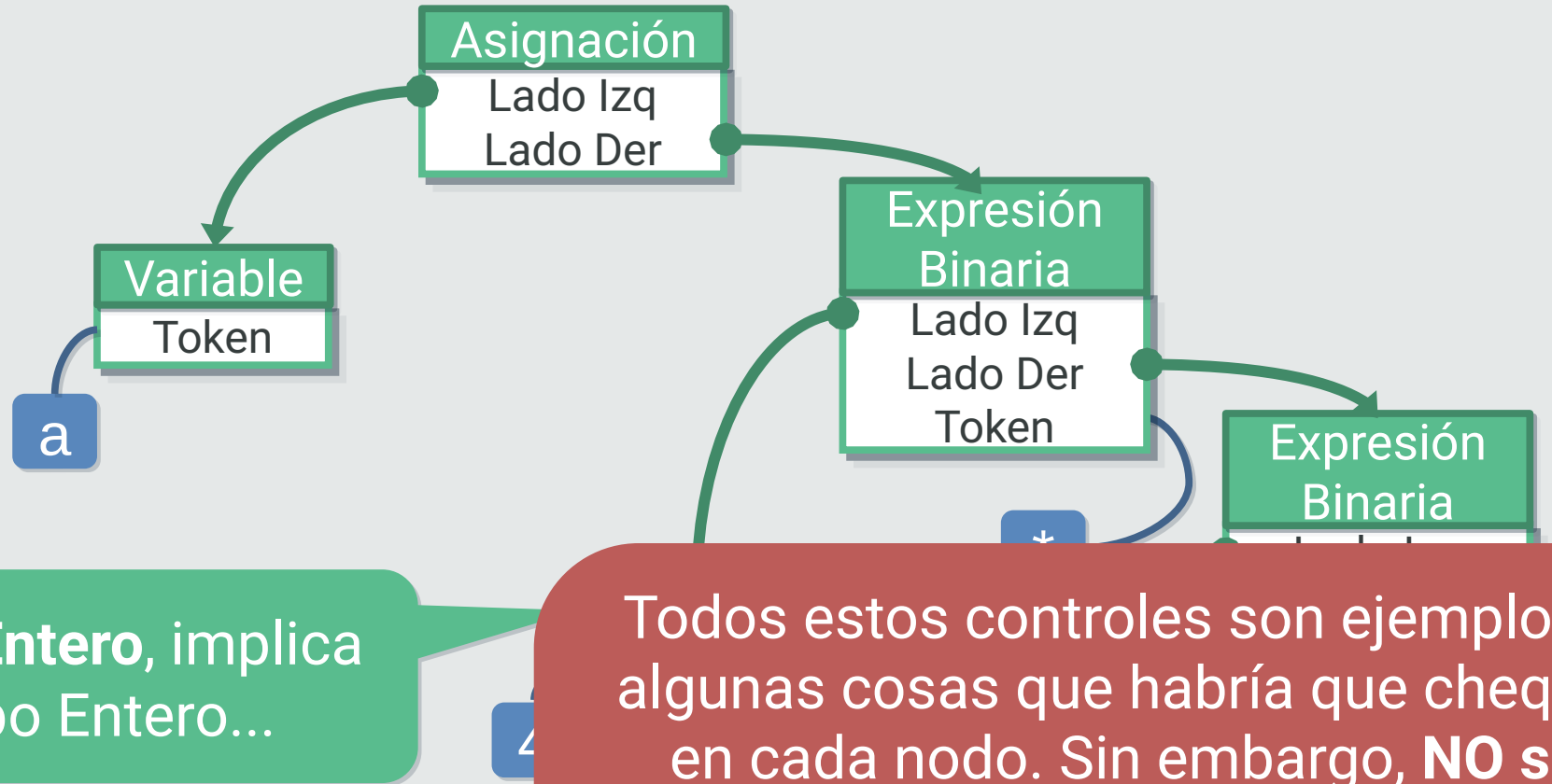
```
…
a = 4*(b+2);
…
```



Chequear un **Entero**, implica retornar tipo Entero...

Chequeo Sentencias – ASTs – Ej. Controles

```
…
a = 4*(b+2);
…
```



Chequear un **Entero**, implica retornar tipo Entero...

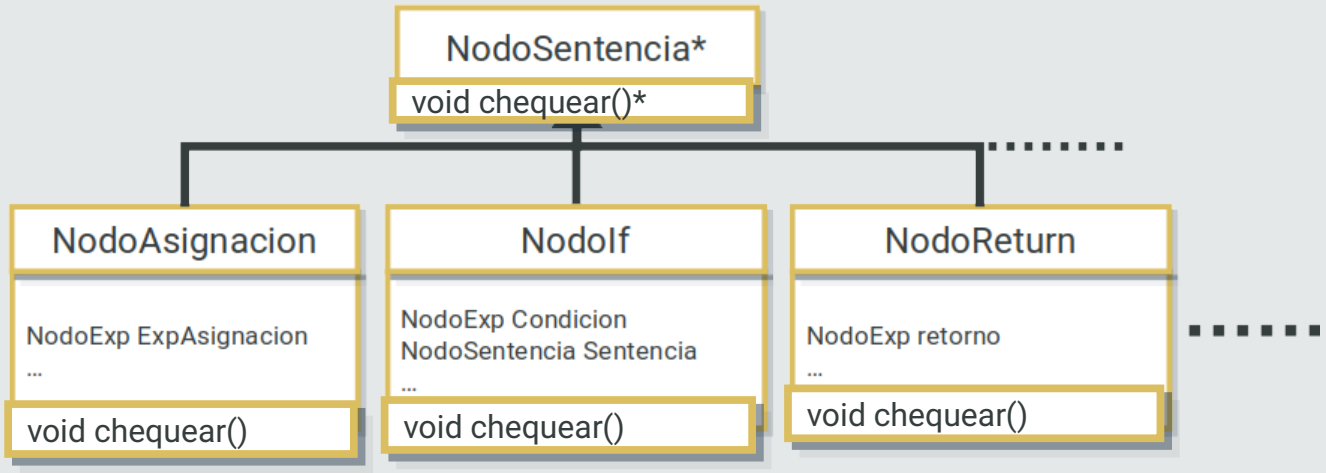
Todos estos controles son ejemplos de algunas cosas que habría que chequear en cada nodo. Sin embargo, **NO son exhaustivos,** es posible que según el lenguaje haya que **controlar mas cosas!!!**

Chequeo Sentencias – ASTs – Implementación

- De forma similar al chequeo de declaraciones, vamos a valernos de los conceptos de **Responsabilidades** y **Delegación**
- Entonces las **claves** para implementar el **chequeo de sentencias** recorriendo el AST es que para **cada nodo**:
  - **Realizar los controles** que le corresponden según al **significado del nodo**
  - **Delegar** el **chequeo de sentencias** a cada uno de sus **hijos**

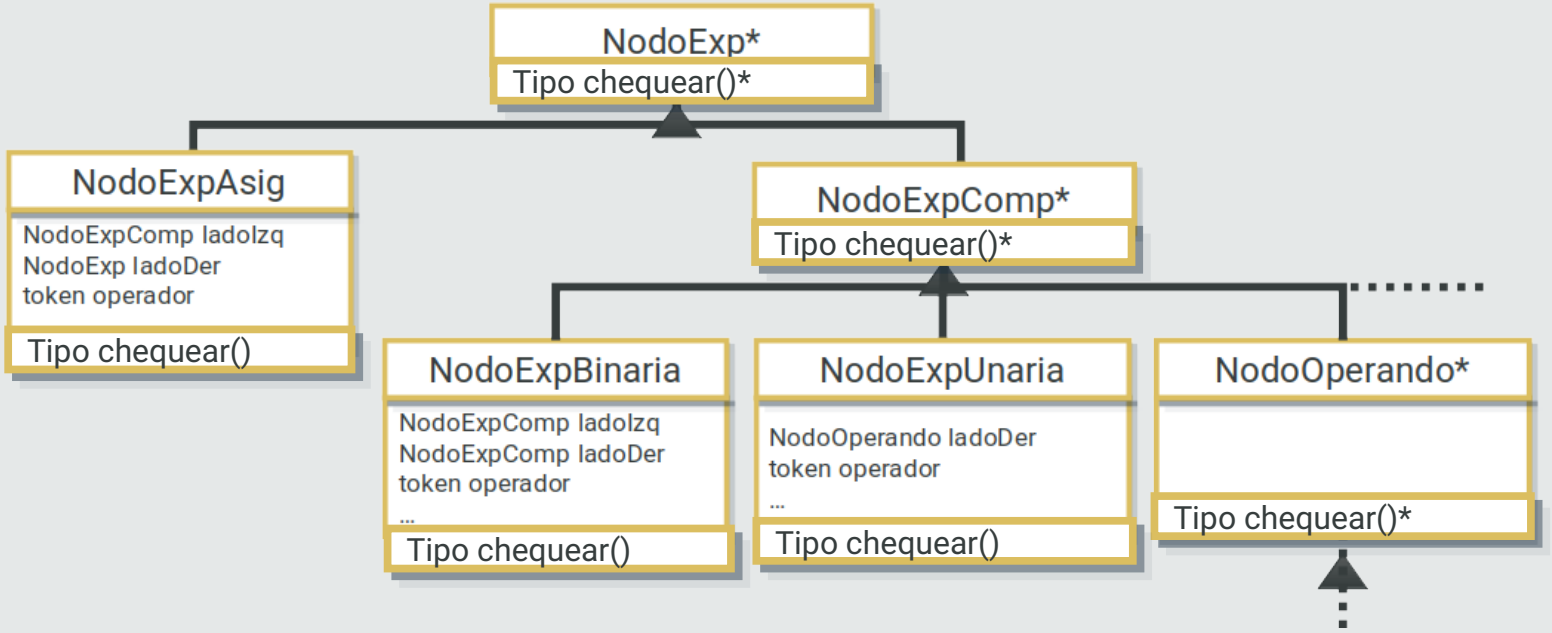
Chequeo Sentencias – ASTs – Implementación

- Una implementación posible:
- Agregamos a cada tipo de nodo del AST un **método** donde implementaremos los **controles asociados al nodo**



Chequeo Sentencias – ASTs – Implementación

- Una implementación posible:
- Agregamos a cada tipo de nodo del AST un **método** donde implementaremos los **controles asociados al nodo**



Chequeo Sentencias – ASTs – Implementación

- Por Ejemplo (Pseudo codigo)

NodoIFSolo
NodoExp condicion NodoSentencia sentenciaThen …
void chequear()

Si *Tipo* de **condicion.chequear()** no es **boolean**
entonces **ERROR!**
sino **sentenciaThen.chequear()**

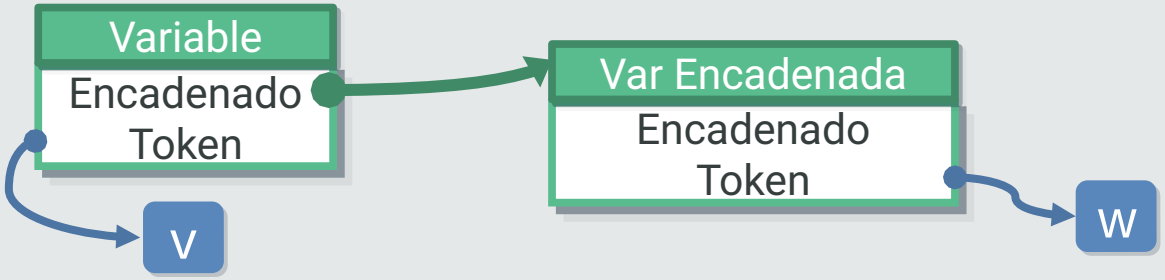
NodoExpAsig
NodoExpComp ladoIzq NodoExp ladoDer token simbolo …
Tipo chequear()

Si el tipo **ladoDer.chequear()** no conforma con el tipo de **ladoIzq.chequear()**
entonces **ERROR!**
…
Retrornar tipo del **ladoIzq**

Chequeo Sentencias – ASTs – Implementación

- En la **asignación** el **nodo padre** (nodoAsignacion) **depende** de los resultados del control de sus **hijos** (lado izq y lado der)
- Existen **situaciones inversas**, donde un nodo **hijo necesita información** de un nodo **padre**
- Por ejemplo: **Llamadas** o acceso a **atributos encadenados**

```
v.x
```

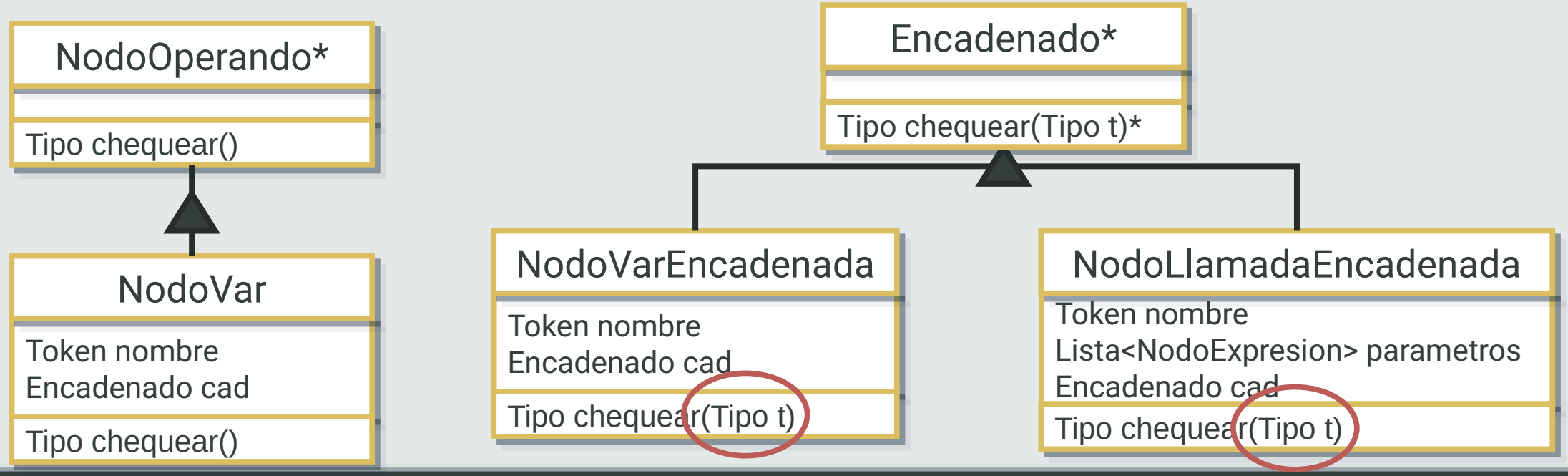


Chequeo Sentencias – ASTs – Implementación

- ¿Cómo podemos **resolver esto**?

Chequeo Sentencias – ASTs – Implementación

- ¿Cómo podemos **resolver esto**?
  - Agregando **parámetros de entrada** en los **métodos de chequeos** de sentencias de los nodos correspondientes



Chequeo Sentencias – ASTs – Implementación

- ¿Cómo se estructura el **chequeo de sentencias** de **todo el programa** en un lenguaje como MiniJava?
- Debemos **controlar el código** de **todas las unidades** (métodos/constructores) del programa fuente
  - Programa esta dividido en clases…
  - Básicamente realizar **chequeo de sentencias** para una **clase** implica realizar el **chequeo de sentencias** para todas sus unidades
  - El **chequeo de sentencias** de un **método** implica realizar el **chequeo del bloque principal** de la unidad

Chequeo Sentencias – ASTs – Implementación

- El **chequeo de sentencias** usa las **entidades declaradas** para:
  - Resolver Nombres
  - Chequear sus tipos
- Las **entidades** deben estar **correctamente declaradas**!!!
  - Por lo tanto debemos realizar **primero el chequeo de declaraciones**



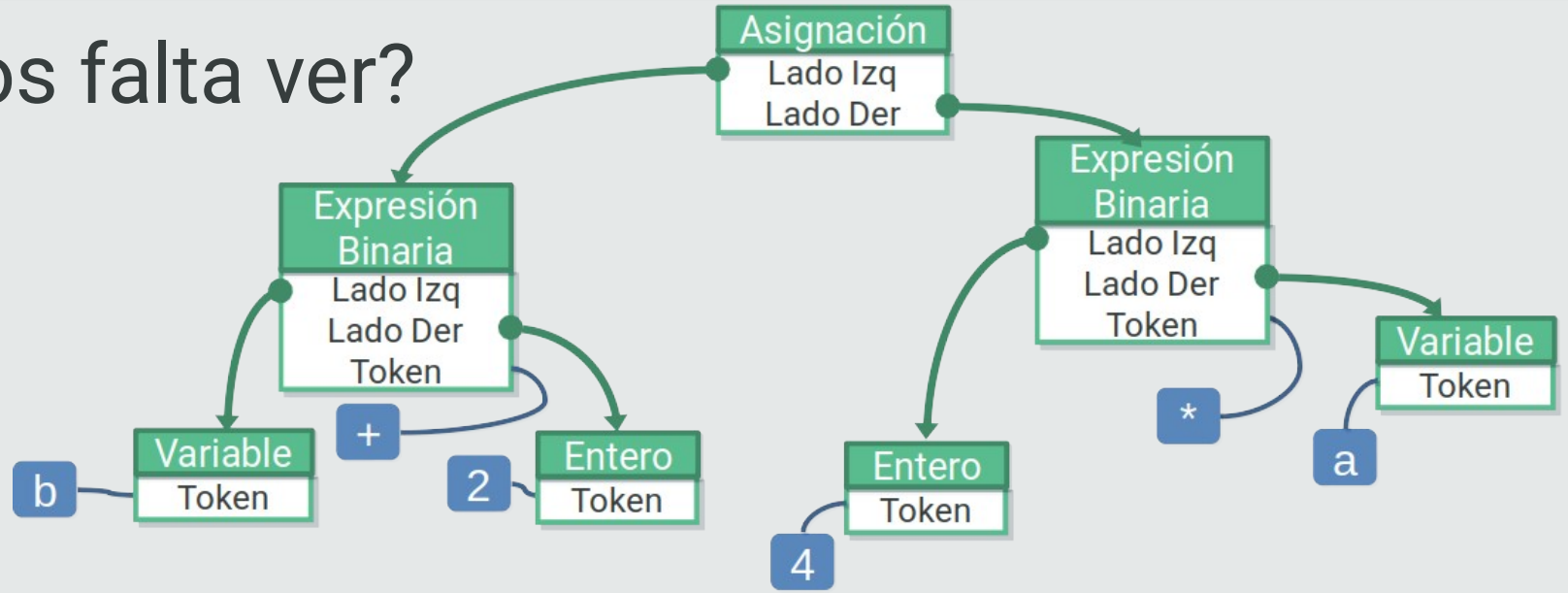
Chequeo Sentencias – ASTs – Implementación

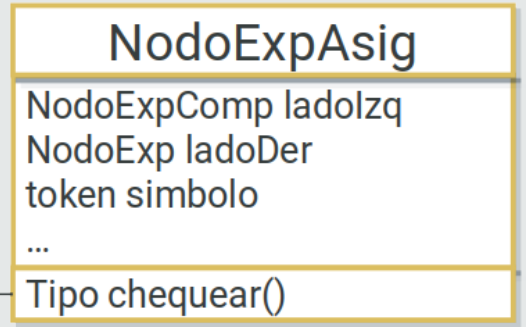
- En los lenguajes como MiniJava donde al **herencia se implementa por copia**, un **método** puede **aparecer** en la tabla de métodos de **mas de una clase**
- ¿Qué **consideración** deberíamos tener en cuenta al realizar el chequeo de sentencias respecto a esto?
  - Solo debemos chequear el cuerpo de un método en la clase donde fue declarado!

Chequeo Sentencias – ASTs – Algo mas?

- Terminamos? Que nos falta ver?

```
…
b+2 = 4*a;
…
```





Si el tipo **ladoDer.chequear()** no conforma con el tipo de **ladoIzq.chequear()**
entonces **ERROR!**
…
Retornar tipo del **ladoIzq**